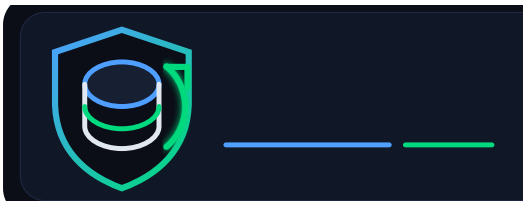

OPERATIONS AND MANAGEMENT HANDBOOK

DBAegis Complete Product Handbook

Main Topics

This table lists the primary handbook topics and the PDF page where each topic starts.

DBAegis Product Operations Handbook

This handbook is the customer-facing guide for installing, upgrading, managing, monitoring, troubleshooting, and recovering DBAegis. It is written as a standalone operations and management book, with product procedures and UI screenshots embedded directly in the PDF.

Use this as the primary operator handbook for day-to-day product management and for planned lifecycle work such as upgrades, downgrades, license renewal, backup validation, and control-plane recovery.

Product Overview

DBAegis is a self-hosted database resilience platform. It provides a web UI and API for registering database connections, running manual backups, creating schedules, restoring artifacts, managing storage destinations, receiving notifications, and validating product/license health.

Core product responsibilities:

- install and run the DBAegis UI/API service on a customer VM
- store product metadata in the local SQLite metadata database
- encrypt saved credentials and sensitive options with `DBAEGIS_SECRET_KEY`
- orchestrate backup and restore jobs through database-native tools, cloud APIs, or database protocols
- preserve backup artifacts under the configured backup directory
- enforce edition entitlements and license limits
- expose locked or upgrade-required UI states when a feature is outside the active edition
- keep release packages separate from runtime data, generated credentials, TLS keys, and customer backup artifacts

DBAegis is not a replacement for database engine HA, replication, cloud provider snapshots, cloud-native PITR, or vendor support. DBAegis can perform engine-specific point-in-time restore where the edition supports physical restore and the operator supplies a valid base backup plus the required copied log/archive chain. Use DBAegis backups as part of a wider recovery plan and validate restores on a schedule.

Editions At A Glance

Area	Community	Professional	Enterprise
License	No license required	Signed license required	Signed license required
Main use	Evaluation, labs, small local workflows	Production team backup/restore	Regulated or large customer deployments
Support	None	Limited	Full
Package	Community package	Signed Professional package	Signed private/offline package
Databases	PostgreSQL, MySQL, MongoDB	Paid database matrix	Paid matrix plus contracted certification

Area	Community	Professional	Enterprise
Backup/restore	Local logical only	Local, cloud, DB-server-local, physical, and PostgreSQL/Oracle PITR where supported	Professional plus contracted certification
Users	1 admin	5 by default	Licensed/unlimited
Connections	3	50 by default	Licensed/unlimited
Schedules	3	100 by default	Licensed/unlimited
Storage types	DBAegis local only	Local, DB-server-local, S3, Azure Blob, GCS	Professional plus enterprise terms
Storage destinations	1 local destination	10 by default	Licensed/unlimited
Retention	Basic	Enabled	Enabled
Notifications	Locked	Email and daily summaries	Email, daily summaries, and webhooks
Access control	Single admin	RBAC and MFA	RBAC, MFA, LDAP / Active Directory
Audit	Locked	Event trail	Event trail and CSV export
Reports	Locked	In-app views only	CSV exports
Self-backup	Locked	Enabled	Enabled
Air-gapped install	No	Optional	Yes

Audit means DBAegis records administrative and API activity in the local metadata database for operator review. Professional and Enterprise include the database-backed audit event trail. Enterprise adds audit CSV export for offline reviews, compliance evidence, or support packages. Audit records include fields such as event type, severity, actor, source IP, object, action, status, message, and scrubbed metadata; sensitive values are redacted before storage. If a paid license token overrides the default feature list, it must include `audit.events` for the audit trail to remain enabled.

Air-gapped install means the customer can deploy from a controlled package without direct public internet access from the DBAegis VM. Community is not positioned for air-gapped delivery. Professional air-gapped delivery is optional and requires the required OS packages, Python packages, database clients, and cloud/vendor tools to be staged through the customer's internal repository or approved media. Enterprise can include a contracted private/offline bundle, checksums, dependency inventory, and customer-scoped installation evidence. No air-gapped package should include customer secrets, runtime data, license issuer private keys, TLS private keys, or customer backup artifacts.

Each customer package is scoped to one edition. Community delivers the Community runtime; Professional adds production backup/restore and team management capabilities; Enterprise adds enterprise integration and reporting capabilities. Customer runtime data, secrets, generated files, and local backup artifacts are not part of a clean release package.

Customer Package Contents

A DBAegis customer package is the installable product distribution for one edition. It contains the application code, web UI, installer, product CLI, dependency constraints, release metadata, and customer documentation needed to install, upgrade, operate, and recover DBAegis on a customer VM. It is not a copy of a running server and it does not contain customer data or customer secrets.

The package is organized around these product areas:

Area	What The Customer Receives	Purpose
Product runtime	`app/`, `bin/`, `conf/`, and edition-specific runtime files for the purchased edition	Runs the API, web UI, backup/restore orchestration, licensing, auth, and operational commands.
Web UI assets	Shared UI files and edition-specific UI files where included	Provides the browser interface for dashboards, connections, storage, schedules, backup history, restore jobs, notifications, reports, and access control.
Installer and lifecycle tools	`bin/install.sh`, `bin/uninstall.sh`, `bin/dbaegis`, `bin/dbaegis-stack`, rollback instructions	Installs, upgrades, rolls back, repairs runtime ownership, starts the service, and removes product files while preserving customer backup artifacts.
Dependency inventory	`requirements/` constraints and dependency inventory	Recreates the tested Python environment and supports customer dependency review.
Customer documentation	Handbook PDF, install/upgrade guide, product edition guide, backup/restore support documentation, and other edition-appropriate docs	Gives operators the procedures to install, manage, troubleshoot, and recover the product.
Release metadata	`release.json`, `PACKAGE_CONTENTS.json`, release notes, and checksums delivered with the release	Identifies the exact edition, version, build identifier, and package contents delivered to the customer.

Every customer tarball also includes these package-root operator files:

```
INSTALL_UPGRADE_UNINSTALL_STEPS.md
PACKAGE_CONTENTS.json
RELEASE_NOTES.md
ROLLBACK.md
release.json
```

`release.json` identifies the packaged edition, release channel, build time, and build identifier. `PACKAGE_CONTENTS.json` is the customer-visible manifest for the exact files in that tarball.

Edition scope:

Edition	Package Scope
Community	Contains the Community runtime, shared product services, installer, CLI, customer handbook, dependency constraints, and shared UI assets. It supports DBAegis-local logical backup and restore for PostgreSQL, MySQL, and MongoDB, with paid features visible only as locked or upgrade-required where applicable.
Professional	Contains everything in Community plus Professional runtime and UI capabilities for production backup/restore, RBAC, MFA, email notifications, cloud/local storage destinations, schedules, retention, audit collection, self-backup, and Professional database support.
Enterprise	Contains everything in Professional plus Enterprise capabilities for LDAP / Active Directory, webhook delivery, CSV report exports, audit export, Enterprise readiness documentation, and webhook security.

The package intentionally excludes customer-owned or environment-specific content:

Excluded Area	Examples	Reason
Customer data and runtime state	metadata DB, generated logs, runtime files, rollback snapshots, temporary staging files	These are created or preserved on the customer VM and must not be overwritten by a release package.
Customer backup artifacts	`/backups`, self-backup archives, database export files, restore staging files	These belong to the customer retention and recovery process.
Secrets and credentials	license tokens, issuer private keys, TLS private keys, `DBAEGIS_SECRET_KEY`, database passwords, cloud credentials, SMTP secrets, LDAP bind credentials, webhook secrets	Secrets must be delivered or stored through the customer-approved secret process, not embedded in product packages.
Build and validation material	Internal build automation, test suites, validation evidence, and local caches	These are used to build and validate releases, but they are not customer runtime dependencies.
Installed runtimes	installed `python/`, `venv/`, and installer-managed vendor tool directories	The installer creates or validates these on the target VM so runtime ownership and platform compatibility are correct.

Common excluded paths:

```
.github/
scripts/
tests/
backups/
data/
license/
logs/
```

```
python/
rollback/
run/
tls/
tmp/
vendor/
venv/
```

Before installing or upgrading, verify the delivered package:

```
sha256sum -c SHA256SUMS
tar tzf EDITION-vVERSION.tar.gz | head
tar xzf EDITION-vVERSION.tar.gz
cd EDITION-vVERSION
cat release.json
cat PACKAGE_CONTENTS.json
```

Use this section with the rest of the handbook: product responsibilities are summarized in Product Overview, edition behavior is defined in Editions At A Glance and Edition Upgrade And Downgrade, installed paths are described in Product File And Folder Structure, and operational procedures are covered in Fresh Install Procedure, Upgrade Procedure, License Operations, Backup and Restore, Monitoring, Troubleshooting, and Control-Plane Recovery.

Pre-Install Prerequisites

Prepare the target VM before running the installer.

Host prerequisites:

- Linux VM with `systemd`
- root or sudo access for install, upgrade, uninstall, service control, package installation, and filesystem ownership repair
- package manager access through `dnf`, `yum`, or `apt`, or an approved offline mirror/bundle for the same packages
- x86_64/amd64 or aarch64/arm64 CPU architecture for the default embedded Python runtime
- time synchronization through NTP or the customer standard time service
- stable hostname, DNS name, or IP address for the operator UI endpoint
- firewall rules for the chosen UI ports and any required outbound destinations

Baseline sizing guidance:

Resource	Guidance
CPU	Start with 2 vCPU for small/lab use; increase for concurrent backup, restore, compression, and cloud transfer workloads.
Memory	Start with 4 GB RAM for small/lab use; increase for large logical exports, compression, restore staging, and multiple concurrent jobs.

Resource	Guidance
Install filesystem	Reserve space for `/opt/dbaegis`, logs, metadata, embedded Python, virtualenv, rollback snapshots, and optional vendor tools.
Backup filesystem	Size `/backups` or the configured backup directory for database size, schedule frequency, retention policy, and restore-drill copies.
Temp filesystem	Size `/opt/dbaegis/tmp` or the configured temp path for the largest backup or restore artifact that may be staged on the DBAegis VM.
Inodes	Monitor inode usage when many small artifacts, logs, or self-backups are retained.

Network prerequisites:

- operator browser access to `3000` for HTTP or `3443` for HTTPS, according to the selected TLS mode
- DBAegis VM access to managed database endpoints and self-managed database VMs
- SSH access from DBAegis to DB VMs when using DB-server-local, physical, or server-side tool modes
- cloud API access for AWS S3, Azure Blob, GCS, or other configured storage destinations
- SMTP access for Professional and Enterprise email notifications
- LDAP / Active Directory access for Enterprise LDAP authentication
- webhook egress for Enterprise webhook notifications

Dependency prerequisites:

- Standard online installs download and checksum-verify the pinned embedded Python runtime, create `/opt/dbaegis/venv`, and install pinned Python dependencies from the package constraints.
- Offline or restricted-network installs must use an approved offline bundle, customer package mirrors, or `DBAEGIS_PYTHON_BIN` / `DBAEGIS_PYTHON_DOWNLOAD=skip` with a preinstalled Python 3.12+ runtime.
- Optional vendor clients such as SnowSQL, SqlPackage, MongoDB tools, ClickHouse client, and database-native physical backup tools are installed only when requested or when the customer provides them through approved software channels.

Customer inputs to collect before install:

- target edition and package version
- VM hostname, UI DNS name, and network allowlist
- install base, metadata DB path, backup directory, and temp directory
- TLS mode and certificate/key paths
- service user name, usually `dbaegis`
- paid-edition license token and public verification key when installing Professional or Enterprise
- database credentials, SSH credentials, cloud storage credentials, SMTP credentials, LDAP bind account, and webhook endpoint secrets as applicable
- off-host location for `DBAEGIS_SECRET_KEY`, self-backups, package checksums, and restore-drill evidence

Runtime Architecture

Default deployment components:

- `systemd` service: `dbaegis`
- backend API: FastAPI process on `127.0.0.1:8000` by default
- web/API front door: DBAegis-managed nginx process launched by `bin/dbaegis-stack`
- HTTP UI port: `3000` by default
- HTTPS UI/API port: `3443` when TLS is enabled
- metadata DB: SQLite at `/opt/dbaegis/data/dbaegis.db` by default
- config file: `/opt/dbaegis/conf/dbaegis.conf`
- normal database backup artifacts: `/backups` by default
- DBAegis self-backups: `/backups/self` by default

Important ownership rules:

- runtime state, config, metadata DB, logs, rollback snapshots, license files, backup directories, and TLS files under the install base are owned by the DBAegis service user where applicable
- installer-managed vendor tools under `/opt/dbaegis/vendor` are owned by the DBAegis service user and readable/executable by the DBAegis service process; stable `/usr/local/bin` wrappers remain root-owned
- never make root-controlled wrapper paths writable by the service user
- never delete `/backups` during product uninstall unless an operator performs a separate retention-approved filesystem cleanup

Default Paths And Ports

Item	Default
Install base	`/opt/dbaegis`
Service user	`dbaegis`
Config	`/opt/dbaegis/conf/dbaegis.conf`
Metadata DB	`/opt/dbaegis/data/dbaegis.db`
Logs	`/opt/dbaegis/logs`
Runtime files	`/opt/dbaegis/run`
Temporary staging	`/opt/dbaegis/tmp`
TLS material	`/opt/dbaegis/tls`
License files	`/opt/dbaegis/license`
Rollback snapshots	`/opt/dbaegis/rollback`

Item	Default
Database backup artifacts	`/backups`
Self-backups	`/backups/self`
HTTP UI	`http://HOST:3000`
HTTPS UI/API	`https://HOST:3443`
Backend API local port	`127.0.0.1:8000`

The backend API port is normally local-only behind nginx. Operators should test through port `3000` for HTTP and `3443` for HTTPS unless they are debugging on the VM itself.

Network, Firewall, And TLS

Default access behavior:

- HTTP is available on `3000` when `TLS_MODE=off` or `HTTP_BEHAVIOR=both`
- HTTPS is available on `3443` when `TLS_MODE=self_signed` or `TLS_MODE=customer_provided`
- backend API port `8000` should stay local to the VM and is normally reached through the UI/API front door

Open only the ports required by the customer design. For a lab or trusted-network VM using HTTP, allow `3000` from trusted operator networks. For production HTTPS, allow `3443` and prefer `HTTP_BEHAVIOR=redirect` or `HTTP_BEHAVIOR=https_only`.

TLS modes:

Mode	Use
`off`	HTTP-only lab or trusted network use
`self_signed`	Lab HTTPS validation; browsers will warn because the issuer is not trusted
`customer_provided`	Production HTTPS with customer certificate, key, and optional chain

A browser warning on `https://HOST:3443` is expected with self-signed certificates or certificates whose hostname does not match the URL. That is a certificate trust issue, not a product defect. For production, install a customer-trusted certificate and use the DNS name in the certificate.

Replace or renew customer-provided TLS material:

```
sudo install -d -m 0750 -o dbaegis -g dbaegis /opt/dbaegis/tls
sudo install -m 0640 -o dbaegis -g dbaegis /tmp/server.key /opt/dbaegis/tls/server.key
sudo install -m 0644 -o dbaegis -g dbaegis /tmp/server.crt /opt/dbaegis/tls/server.crt
sudo install -m 0644 -o dbaegis -g dbaegis /tmp/chain.crt /opt/dbaegis/tls/chain.crt
sudo systemctl restart dbaegis
curl -k -I https://127.0.0.1:3443/
```

Then verify the public hostname from an operator workstation with a browser or TLS scanner trusted by the customer.

Product File And Folder Structure

Source and release package layout:

Path	Purpose
`app/community/`	Community runtime implementation
`app/professional/`	Professional features, including paid signed-license verification, included only in Professional and Enterprise packages
`app/enterprise/`	Enterprise features included only in Enterprise packages
`app/services/`	Shared backup, restore, notification, and service helpers used across editions
`app/main.py`	Runtime dispatcher that loads the correct edition runtime
`app/license.py`	Community-safe license status, edition entitlement, and CLI facade
`bin/install.sh`	Fresh install, upgrade, rollback, and runtime repair installer
`bin/uninstall.sh`	Safe and purge uninstall workflow
`bin/dbaegis`	Product CLI entrypoint
`bin/dbaegis-stack`	Service launcher that renders nginx runtime config and starts UI/API
`conf/dbaegis.conf.example`	Example configuration template
`docs/`	Customer-facing product documentation included with the edition package
`requirements/`	Dependency inventory and install constraints
`ui/`	Shared web UI assets
`ui/professional/`	Professional UI files when present in paid packages
`ui/enterprise/`	Enterprise UI files when present in Enterprise packages
`release.json`	Package manifest with edition, version, build time, and build identifier

Installed runtime layout:

Path	Owner Policy	Purpose
`/opt/dbaegis/conf`	service user	Active product config

Path	Owner Policy	Purpose
`/opt/dbaegis/data`	service user	SQLite metadata DB
`/opt/dbaegis/license`	service user	License token and public verification key
`/opt/dbaegis/logs`	service user	API, backup/restore, scheduler, and nginx logs
`/opt/dbaegis/run`	service user	Runtime process files created by the service
`/opt/dbaegis/rollback`	service user	Pre-upgrade snapshots
`/opt/dbaegis/tls`	service user where under install base	Web TLS certificate/key material
`/opt/dbaegis/tmp`	service user	Temporary backup/restore staging
`/opt/dbaegis/vendor`	service user	Installer-managed vendor executable tools
`/opt/dbaegis/venv`	service user	Installed Python virtual environment
`/backups`	service user	Customer database backup artifacts

Customer release tarballs exclude runtime directories such as `data/`, `license/`, `logs/`, `python/`, `rollback/`, `run/`, `tls/`, `tmp/`, `vendor/`, and `venv/`. The installer creates or repairs those paths on the target VM.

Fresh Install Procedure

Before installing, complete this customer worksheet:

Item	Decision
Edition	Community, Professional, or Enterprise
VM hostname / IP	Public UI hostname and private management IP
Install base	Default `/opt/dbaegis` unless customer policy requires another path
Backup directory	Default `/backups`; must have enough space and must not be removed by uninstall
HTTP / HTTPS behavior	Default HTTP UI on `3000`, HTTPS on `3443` when TLS is enabled
TLS certificate	Self-signed for lab only; trusted customer certificate for production
Bootstrap admin	Generated password or supplied `BOOTSTRAP_ADMIN_PASSWORD`

Item	Decision
License files	Required for Professional and Enterprise before normal paid feature use
Off-host recovery	Location where self-backups and `DBAEGIS_SECRET_KEY` are stored

Verify the package before install:

```
sha256sum -c SHA256SUMS
tar tzf EDITION-vVERSION.tar.gz | head
```

The expected package root is `community-vVERSION`, `professional-vVERSION`, or `enterprise-vVERSION`. Do not install from a directory that contains runtime state such as `data/`, `license/`, `logs/`, `tls/`, `vendor/`, or `venv/`.

1. Copy or extract the correct edition release package on the target VM.

```
cd /tmp
tar xzf EDITION-vVERSION.tar.gz
cd EDITION-vVERSION
```

1. Run a fresh install as root.

```
sudo DBAEGIS_USER=dbaegis bash bin/install.sh --fresh
```

1. For custom paths or TLS, pass installer variables during the fresh install.

```
sudo DBAEGIS_USER=dbaegis \
  DBAEGIS_BASE=/opt/dbaegis \
  DBAEGIS_BACKUP_DIR=/backups \
  DBAEGIS_TLS_MODE=customer_provided \
  DBAEGIS_TLS_CERT_PATH=/etc/pki/dbaegis/server.crt \
  DBAEGIS_TLS_KEY_PATH=/etc/pki/dbaegis/server.key \
  DBAEGIS_TLS_CHAIN_PATH=/etc/pki/dbaegis/chain.crt \
  bash bin/install.sh --fresh
```

1. Verify service status and health.

```
sudo systemctl status dbaegis --no-pager
curl http://127.0.0.1:8000/health
curl http://127.0.0.1:8000/api/version
```

1. Open the UI.

```
http://HOST:3000/
https://HOST:3443
```

1. Capture the bootstrap admin credentials printed by the installer. Fresh installs generate a unique admin password unless `BOOTSTRAP_ADMIN_PASSWORD` was provided. Change or store that password according to the customer secret handling process.
2. For Professional or Enterprise, install the issued license token and public verification key under `/opt/dbaegis/license`, restart the service, and verify license status.

```
sudo systemctl restart dbaegis
curl http://127.0.0.1:8000/api/license/status
```

1. Complete the customer acceptance checks before handing over the VM.

```
curl http://127.0.0.1:8000/health
curl http://127.0.0.1:8000/api/version
curl http://127.0.0.1:8000/api/license/status
sudo journalctl -u dbaegis -n 100 --no-pager
```

Then log in to the UI, change or vault the bootstrap password, create a non-production test connection, run a connection precheck, test storage access, run a small manual backup, run a restore drill to a safe target, and create a self-backup before enabling production schedules.

Upgrade Procedure

Use `--upgrade` for version upgrades and edition package changes. Upgrade preserves `dbaegis.conf`, the SQLite metadata DB, users, roles, schedules, storage destinations, notification settings, self-backup metadata, restore history, audit history, and configured backup artifacts.

1. Extract the new release package.

```
cd /tmp
tar xzf EDITION-vVERSION.tar.gz
cd EDITION-vVERSION
```

1. Take safety copies.

```
sudo -u dbaegis cp -a /opt/dbaegis/conf/dbaegis.conf \
  /opt/dbaegis/conf/dbaegis.conf.pre-upgrade.$(date +%Y%m%d-%H%M%S)

sudo -u dbaegis cp -a /opt/dbaegis/data/dbaegis.db \
  /opt/dbaegis/data/dbaegis.db.pre-upgrade.$(date +%Y%m%d-%H%M%S)
```

1. Run the upgrade.

```
sudo DBAEGIS_USER=dbaegis bash bin/install.sh --upgrade
```

1. Verify the package metadata, service, and UI.

```
curl http://127.0.0.1:8000/api/version
curl http://127.0.0.1:8000/api/license/status
```

```
sudo systemctl status dbaegis --no-pager
curl -I http://127.0.0.1:3000/
```

1. If TLS is enabled, verify HTTPS.

```
curl -k -I https://127.0.0.1:3443/
```

1. Smoke-test login, dashboard, connections, manual backup, restore visibility, schedules, notifications, and any feature affected by the upgrade.

If the UI shows a new action but the API returns `Not Found`, restart `dbaegis.service` and refresh the browser. That usually means the browser loaded new UI assets while the backend process was still running old routes.

Customer upgrade checklist:

1. Confirm the active package and edition with `/api/version`.
2. Confirm the active license with `/api/license/status`.
3. Take config and metadata safety copies.
4. Confirm there is enough disk space for the new package and rollback snapshot.
5. Pause schedules if the maintenance window must avoid new backup starts.
6. Run the target package with `--upgrade`.
7. Verify login, dashboard, backup history, restore jobs, storage destinations, schedules, notifications, access control, and self-backups.
8. Run one non-destructive backup/restore smoke in the active edition.
9. Re-enable schedules and monitor the next scheduled run.

Edition Upgrade And Downgrade

Changing editions is a package operation plus a license operation. Do not change editions by editing only `DBAEGIS_EDITION` in `dbaegis.conf`.

Supported paths:

- Community to Professional
- Professional to Enterprise
- Community directly to Enterprise
- Enterprise to Professional
- Professional to Community
- Enterprise directly to Community

Upgrade from Community to Professional:

```
tar xzf professional-vVERSION.tar.gz
cd professional-vVERSION
sudo install -o dbaegis -g dbaegis -m 0640 customer.license
/opt/dbaegis/license/dbaegis.license
sudo install -o dbaegis -g dbaegis -m 0644 license_public.pem
```

```
/opt/dbaegis/license/license_public.pem
sudo DBAEGIS_USER=dbaegis bash bin/install.sh --upgrade
```

Upgrade from Professional to Enterprise:

```
tar xzf enterprise-vVERSION.tar.gz
cd enterprise-vVERSION
sudo install -o dbaegis -g dbaegis -m 0640 enterprise.license
/opt/dbaegis/license/dbaegis.license
sudo install -o dbaegis -g dbaegis -m 0644 license_public.pem
/opt/dbaegis/license/license_public.pem
sudo DBAEGIS_USER=dbaegis bash bin/install.sh --upgrade
```

Downgrade from Enterprise to Professional:

```
tar xzf professional-vVERSION.tar.gz
cd professional-vVERSION
sudo install -o dbaegis -g dbaegis -m 0640 professional.license
/opt/dbaegis/license/dbaegis.license
sudo install -o dbaegis -g dbaegis -m 0644 license_public.pem
/opt/dbaegis/license/license_public.pem
sudo DBAEGIS_USER=dbaegis bash bin/install.sh --upgrade
```

Downgrade to Community:

```
tar xzf community-vVERSION.tar.gz
cd community-vVERSION
sudo DBAEGIS_USER=dbaegis bash bin/install.sh --upgrade
```

After a downgrade, protected configuration is preserved but locked. For example, self-backups, notifications, storage destinations, reports, LDAP, roles, or webhooks may remain visible as locked or upgrade-required so operators can understand what exists without executing features outside the active edition.

Edition-change verification:

```
curl http://127.0.0.1:8000/api/version
curl http://127.0.0.1:8000/api/license/status
sudo -u dbaegis grep -E '^(DBAEGIS_EDITION|DBAEGIS_LICENSE_REQUIRED|DBAEGIS_LICENSE_DIR)='
/opt/dbaegis/conf/dbaegis.conf
```

Expected behavior by target edition:

Target edition	License requirement	Customer-visible behavior
Community	No paid license required	Community database support remains active; paid sections stay visible as locked or upgrade-required where useful for consistency.
Professional	Professional token required	RBAC, MFA, email notifications, cloud/local storage, schedules,

Target edition	License requirement	Customer-visible behavior
		retention, and self-backup workflows are available within Professional limits.
Enterprise	Enterprise token required	Professional features plus LDAP, webhooks, CSV report exports, audit export, and Enterprise database/support entitlements are available.

Rollback Procedure

The installer creates pre-upgrade snapshots under `/opt/dbaegis/rollback`. Rollback restores application/UI/bin/venv runtime files and the systemd unit from a prior snapshot while preserving active `dbaegis.conf`, SQLite metadata, and backup artifacts.

```
sudo DBAEGIS_USER=dbaegis bash /opt/dbaegis/bin/install.sh --rollback
sudo systemctl status dbaegis --no-pager
curl http://127.0.0.1:8000/health
```

Use rollback when an upgrade breaks service startup or core UI/API behavior. If metadata migrations were involved, validate login, license, schedules, and restore visibility after rollback.

Uninstall Procedure

Safe uninstall removes product runtime/service files but preserves customer state and backup artifacts.

```
sudo bash /opt/dbaegis/bin/uninstall.sh --yes
```

Purge uninstall removes additional DBAegis config/runtime paths but still preserves backup artifact directories such as `/backups`.

```
sudo bash /opt/dbaegis/bin/uninstall.sh --purge --yes
```

Uninstall does not delete `/backups`, the configured `BACKUP_DIR`, or a separate `SELF_BACKUP_DIR`. Those paths may contain customer database backups and system backup snapshots. Delete backup directories only through an explicit operator-controlled retention process after confirming off-host copies.

Uninstall verification:

```
systemctl status dbaegis --no-pager
test -d /backups && sudo du -sh /backups
test -d /backups/self && sudo du -sh /backups/self
```

After safe uninstall, reinstall with `bin/install.sh --fresh` or `--upgrade` from the target package and the preserved config/metadata can be reused. After purge uninstall, the next install behaves like a new product install unless the operator restores metadata/config from a self-backup or safety copy.

Day-To-Day Product Management

Common service commands:

```
sudo systemctl status dbaegis --no-pager
sudo systemctl restart dbaegis
sudo journalctl -u dbaegis -n 100 --no-pager
sudo journalctl -u dbaegis --since "2 hours ago" --no-pager
```

Common local health checks:

```
curl http://127.0.0.1:8000/health
curl http://127.0.0.1:8000/api/version
curl http://127.0.0.1:8000/api/license/status
curl -k https://127.0.0.1:3443/health
```

Config changes:

- edit `/opt/dbaegis/conf/dbaegis.conf`
- preserve `DBAEGIS_SECRET_KEY`
- restart `dbaegis`
- verify health and login

Do not edit runtime nginx files under `/opt/dbaegis/run/nginx/conf`. `bin/dbaegis-stack` renders those files from `dbaegis.conf` each time the service starts.

UI Operations Screens

Use these screens when training operators or validating a customer deployment. The customer handbook PDF includes the current DBAegis UI screenshots at desktop width. Paid-edition controls that are visible in Community appear as locked or upgrade-required. Use the notes below with DBAEGIS_HANDBOOK.pdf during operator training and deployment validation.

Login

The login screen is the normal entry point for local users and, in Enterprise, LDAP users when LDAP is configured. Operators should confirm the correct public HTTP or HTTPS endpoint, confirm the branded DBAegis logo appears above the sign in form, sign in with an active account, and verify that license or MFA prompts appear only when expected.

Dashboard

The dashboard gives operators a first health view after login. Use it after install, upgrade, restore, or license changes to confirm the service is responding and that the visible edition state matches the installed package and license.

Connections

The connections page is where database endpoints are created, tagged, checked, and managed. Operators should use connection prechecks before scheduling backups or attempting restores, and should keep connection tags aligned with RBAC assignments where roles are used.

Storage

The storage page manages backup destinations. Community is limited to DBAegis local storage, while Professional and Enterprise can use DB-server-local and cloud destinations when licensed. Confirm write access and retention behavior before using a destination for production schedules.

Schedules

The schedules page controls recurring backup jobs. Operators should create schedules only after the connection and storage destination are validated, and should review schedule lag, failure history, and retention settings regularly.

Backup History

Backup history is the operational record of completed and failed backups. Use it to confirm artifact location, backup type, storage target, retention outcomes, and the source backup ID needed for restore workflows.

Failed backup entries can be dismissed after review. Dismissal keeps the failed backup row, logs, artifact metadata, and dismiss audit fields in history, but removes that item from dashboard needs-attention and Backup Failure Summary views. Use dismissal for reviewed failures that no longer need active operator attention; use delete/cancel only when intentionally removing the history row and associated artifact. The backup Retry action starts a new backup for the same connection using current settings and does not rerun the old failed row.

Restore

The restore page is where operators select backup artifacts and target connections. Always validate the target, use a non-production destination for drills, and preserve the source artifact until restore validation is complete.

Reports

Reports summarize backup, restore, schedule, and audit activity according to the active edition. Enterprise unlocks CSV export workflows for customers who need offline reporting, audit packages, or operational review evidence.

Notifications

The notifications page manages email and, in Enterprise, webhook delivery. After changing SMTP or webhook settings, run a test notification and confirm the result before relying on the channel for backup or restore alerts.

System Backups

System backups protect the DBAegis control-plane metadata and configuration state. They do not replace managed database backups. Use this page to create, review, and retain self-backups before upgrades or major configuration changes.

Access Control Users

The Users tab is used to create, disable, and manage local users. DBAegis prevents removing the last active admin, and operators should keep a local break-glass admin even when Enterprise LDAP is enabled.

Access Control Roles

The Roles tab is used for RBAC in Professional and Enterprise. Assign roles by responsibility and, where needed, constrain them by connection or tag so backup and restore operators see only the systems they manage.

Access Control LDAP

The LDAP tab is Enterprise-only. Use it to configure LDAP / Active Directory authentication and group-to-role mapping. Lower editions keep the panel visible as locked so operators understand the upgrade path.

Access Control MFA

The MFA tab controls local-user TOTP enrollment in Professional and Enterprise. MFA secrets are encrypted with the DBAegis secret key, and disabling global MFA bypasses existing enrollments without deleting them.

Users, Roles, MFA, And LDAP

User management rules:

- Community supports one local admin user
- Professional supports 5 users by default
- Enterprise supports licensed or unlimited users
- DBAegis prevents disabling, deleting, or demoting the last active admin
- each user is limited to one active login session when single-session controls are enabled

Role and access-control rules:

- Professional and Enterprise support RBAC and custom roles
- built-in scoped operator roles include `backup_operator`, `restore_operator`, and `db_operator`
- roles can be constrained by database connection or connection tag
- LDAP users inherit read-only roles from LDAP group mappings

MFA rules:

- MFA is locked in Community
- Professional and Enterprise support local-user TOTP MFA
- MFA works with Microsoft Authenticator and other standard authenticator apps
- MFA secrets are encrypted with `DBAEGIS_SECRET_KEY`
- disabling global MFA bypasses existing enrollments without deleting them

LDAP rules:

- LDAP / Active Directory is Enterprise-only

- LDAP settings, LDAP login, and LDAP group-to-role mappings remain blocked in Community and Professional
- keep local break-glass admin access even when LDAP is configured

Local admin password recovery:

- use this only from the DBAegis VM shell for local admin users
- LDAP-managed users must be reset in LDAP / Active Directory
- the reset clears sessions only for the selected local admin
- generated passwords are printed once and are not written to `dbaegis.conf`

```
sudo -u dbaegis /opt/dbaegis/bin/dbaegis reset-admin-password --dry-run
sudo -u dbaegis /opt/dbaegis/bin/dbaegis reset-admin-password --generate
sudo -u dbaegis /opt/dbaegis/bin/dbaegis reset-admin-password --username admin
```

If the only local break-glass admin was disabled, reset and reactivate it:

```
sudo -u dbaegis /opt/dbaegis/bin/dbaegis reset-admin-password --username admin --activate --
generate
```

Connections, Storage, Backups, And Restores

Connection setup checklist:

1. Select the database type allowed by the active edition.
2. Enter host, port, database/service name, username, and password or token.
3. Add SSH details when using DB-server-local or server-side tool paths.
4. Add database TLS/certificate paths when required by the database.
5. Run connection precheck.
6. Save tags if role-based access or schedule grouping needs them.

Storage setup rules:

- Community supports DBAegis local storage only
- Professional supports local, DB-server-local, AWS S3, Azure Blob, and GCS
- Enterprise supports Professional storage plus contracted support terms
- cloud storage credentials are encrypted and redacted
- retention may delete local or remote artifacts when configured

Backup rules:

- logical backup is normally best for migrations and version changes
- physical backup is normally best for same-version disaster recovery
- PostgreSQL point-in-time restore requires a physical base backup and a complete copied WAL chain that covers the target time
- Oracle RMAN point-in-time restore requires a physical RMAN backup plus archived redo logs from the backup or a copied archive-log source
- DB-server-local modes require SSH and database tools on the database server

- DBAegis-local and cloud modes run tools from the DBAegis VM unless the mode says otherwise
- optional vendor tools such as SnowSQL, SqlPackage, MongoDB tools, and ClickHouse client are installed only when requested by installer flags
- MySQL and MariaDB physical backups should use `xtrabackup`, `mariadb-backup`, or `mariabackup` whenever possible. If a configured datadir path is used instead, the path must be an offline copy or frozen filesystem snapshot. DBAegis blocks paths that look like live InnoDB or MariaDB data directories unless `physical_options.consistency` is explicitly set to `snapshot-safe`; do not use that setting for a live, changing datadir.
- SQL Server and Azure SQL logical BACPAC operations use `sqlpackage`. Passwordless or token-based authentication is preferred. DBAegis blocks legacy SQL password arguments by default because they expose the password in the process list; enable `DBAEGIS_ALLOW_SQLPACKAGE_PASSWORD_ARG=1` only when the customer explicitly accepts that operational risk.

Restore rules:

- always confirm target connection and destination before restore
- use a non-production target for validation drills
- for PostgreSQL PITR, choose physical restore, set `target_pgdata`, set `pitr_target_time` or `recovery_target_time`, and provide `pitr_log_source` or `wal_source` when the WAL source was not already saved with the backup
- for Oracle RMAN PITR, choose physical restore, set `pitr_target_time` or `pitr_target_scn`, and provide `pitr_log_source` or `archive_log_source` when the selected RMAN backup did not include the required archived redo logs
- verify restore history, job logs, row/object counts, and application-level behavior after restore
- preserve source artifacts until the restore is validated

PostgreSQL copied-log PITR workflow:

1. Run a PostgreSQL physical backup. DBAegis records the base backup in backup history and creates a PITR chain catalog row.
2. Keep the WAL/archive log directory or copied WAL backups for the full recovery window. A base backup alone can restore only to the backup state.
3. Start a physical restore from the base backup.
4. Enter the target PGDATA directory.
5. Enter the desired target timestamp in `pitr_target_time` or `recovery_target_time`.
6. Enter the copied WAL directory in `pitr_log_source` or `wal_source` if the backup catalog row does not already include it.
7. Leave `recovery_target_action` blank for DBAegis to promote the recovered database automatically, or set it to `pause` or `shutdown` when the recovery drill requires PostgreSQL's alternate target behavior.
8. Validate the restored PostgreSQL instance before using it for production.

Oracle RMAN PITR workflow:

1. Run an Oracle physical RMAN backup with `Include archive logs` enabled when the archived redo logs should be packaged with the backup.

2. If archive logs are managed separately, keep the copied archive-log directory for the full recovery window.
3. Start a physical restore from the RMAN backup.
4. Enter the target timestamp in `pitr_target_time` or the target SCN in `pitr_target_scn`. The accepted formats are `YYYY-MM-DD HH:MM:SS`, `SCN 1234567`, or `1234567`. Oracle RMAN evaluates timestamp targets in the Oracle database/server time context, so use the database server timestamp for customer restore drills.
5. Enter `pitr_log_source` or `archive_log_source` only when the backup catalog row requires an external archive-log source.
6. DBAegis catalogs the RMAN pieces, writes the RMAN `UNTIL` target inside the RMAN `RUN` block, restores archived logs, recovers the database, and opens with `RESETLOGS` unless validate-only mode is selected.
7. Validate the restored Oracle database before using it for production.

Provider-managed PostgreSQL services such as Amazon RDS, Amazon Aurora, Google Cloud SQL, and Azure Database for PostgreSQL should continue to use provider snapshots and provider-native PITR for disaster recovery. DBAegis uses logical backup/restore for those managed endpoints because provider host files and WAL archives are not exposed to DBAegis.

For production change control, record each customer PITR drill in the customer's own validation evidence with the source backup ID, requested target time or SCN, restored target, validation queries, and operator approval.

Schedules And Retention

Schedule management:

- create schedules only for validated connections
- use clear names that identify database, environment, storage, and frequency
- set retention explicitly for production schedules
- verify schedule lag and failed runs in monitoring
- pause schedules before destructive maintenance windows

Retention behavior:

- manual backup retention defaults to disabled unless an operator enters non-zero values
- schedule retention runs after backup completion
- local retention deletes tracked local artifacts and history rows
- cloud retention deletes tracked remote objects and history rows
- self-backup retention is separate from database backup retention

Notifications, Reports, And Audit

Notifications:

- Professional enables email notifications
- Enterprise enables email and webhooks

- webhook settings are Enterprise-only and remain locked in lower editions
- SMTP passwords and webhook secrets are encrypted with `DBAEGIS_SECRET_KEY`
- always send a test email after changing SMTP settings

Reports:

- in-app backup, restore, and schedule views are available according to edition
- CSV report exports are Enterprise-only
- audit event storage is Professional and Enterprise and uses the local `audit_events` metadata table
- audit entries include actor, source IP, object, action, status, message, and scrubbed metadata for admin/API activity
- a paid license token with an explicit custom feature list must include `audit.events`; otherwise audit storage and the audit-events API are locked
- audit CSV export is Enterprise-only and is intended for offline review, compliance evidence, or support packages

Air-gapped install:

- Community is not positioned for air-gapped delivery
- Professional air-gapped delivery is optional and depends on staged internal OS/Python/database-client repositories or approved installation media
- Enterprise can include a contracted private/offline bundle, checksums, dependency inventory, and customer-scoped validation evidence
- customer data, runtime configuration, TLS keys, license issuer private keys, and backup artifacts must stay outside release packages

If a notification test fails with CSRF or authentication errors, refresh the UI session, retry from the same browser session, then verify `/api/auth/me` returns a current CSRF token and restart the service if backend routes were recently upgraded.

License Operations

Professional and Enterprise require signed offline license tokens. Community does not require a license by default.

License file defaults:

- token: `/opt/dbaegis/license/dbaegis.license`
- public key: `/opt/dbaegis/license/license_public.pem`

Verify a license:

```
/opt/dbaegis/bin/dbaegis license verify \
  --license-key-file /opt/dbaegis/license/dbaegis.license \
  --public-key-file /opt/dbaegis/license/license_public.pem

curl http://127.0.0.1:8000/api/license/status
```

License renewal:

1. Issue a replacement token from the trusted issuer host.
2. Install the token and public key on the customer VM.
3. Restart DBAegis if files changed while the service was running.
4. Verify UI banner, `/api/license/status`, and licensed feature access.

Install or update a paid-edition license token:

```
sudo install -d -m 0750 -o dbaegis -g dbaegis /opt/dbaegis/license
sudo install -m 0644 -o dbaegis -g dbaegis /tmp/license_public.pem
/opt/dbaegis/license/license_public.pem
sudo install -m 0640 -o dbaegis -g dbaegis /tmp/customer.license
/opt/dbaegis/license/dbaegis.license

/opt/dbaegis/bin/dbaegis license verify \
  --license-key-file /opt/dbaegis/license/dbaegis.license \
  --public-key-file /opt/dbaegis/license/license_public.pem

sudo systemctl restart dbaegis
curl http://127.0.0.1:8000/api/license/status
```

Use this process for license renewal, replacing an expired token, moving from a trial token to a production token, changing user or connection limits, or updating paid-edition entitlements without changing the installed edition package. If the license is host-bound, keep `DBAEGIS_LICENSE_INSTANCE_ID` stable across renewals and VM rebuilds.

License expiry UI behavior:

- the UI stays quiet for valid licenses until the license is within 60 calendar days of expiry
- 60 to 16 days left shows a warning banner
- 15 to 8 days left shows an urgent warning banner
- 7 days through the expiry date shows a critical warning banner
- warnings count down the days remaining each day
- expired or invalid paid licenses block protected paid paths
- health, history, restore visibility, and license status remain available so operators can recover

Never store the private license issuer key in a product package or on customer VMs. Keep issuer keys in the trusted private issuer environment only.

Secret Key Management

`DBAEGIS_SECRET_KEY` protects saved connection passwords, storage credentials, SMTP passwords, LDAP bind passwords, MFA enrollment secrets, webhook secrets, and sensitive restore options.

Rules:

- preserve `DBAEGIS_SECRET_KEY` across upgrades and restores
- do not replace it directly in `dbaegis.conf`
- rotate it with `bin/dbaegis rotate-secret-key`
- keep old keys in an operator-controlled secret store if old self-backups may need to be restored

Generate and apply a new key:

```
sudo systemctl stop dbaegis
sudo -u dbaegis /opt/dbaegis/bin/dbaegis rotate-secret-key --generate-new-key --update-conf
sudo systemctl start dbaegis
```

Dry-run before a controlled rotation:

```
sudo -u dbaegis /opt/dbaegis/bin/dbaegis rotate-secret-key --dry-run
```

If the old key is lost, encrypted saved secrets cannot be recovered and must be re-entered.

Self-Backup And Control-Plane Recovery

Self-backups protect DBAegis metadata/config state. They do not replace managed database backups.

Self-backup rules:

- Professional and Enterprise can create and manage self-backups
- Community keeps existing self-backup metadata visible as locked after downgrade
- default location is `/backups/self`
- archived `dbaegis.conf` redacts sensitive values such as `DBAEGIS_SECRET_KEY`
- keep active and retired secret keys outside DBAegis if older self-backups may need recovery

Restore a self-backup:

```
sudo -u dbaegis ls -lh /backups/self/selfbackup_*.zip
sudo systemctl stop dbaegis
sudo -u dbaegis cp -a /opt/dbaegis/data/dbaegis.db \
  /opt/dbaegis/data/dbaegis.db.pre-self-restore.$(date +%Y%m%d-%H%M%S)
tmpdir="$(sudo -u dbaegis mktemp -d /opt/dbaegis/tmp/selfrestore.XXXXXX)"
sudo -u dbaegis unzip /backups/self/selfbackup_YYYYMMDD_HHMMSS_NNNNNNNNN.zip -d "$tmpdir"
sudo -u dbaegis install -m 640 "$tmpdir/dbaegis.db" /opt/dbaegis/data/dbaegis.db
sudo -u dbaegis rm -rf "$tmpdir"
sudo systemctl start dbaegis
```

After restore, validate login, license, saved connections, schedules, notification settings, storage destinations, self-backups, and restore history. If the snapshot was created before a secret-key rotation, use the old key or rotate the restored DB forward while DBAegis is stopped.

Troubleshooting

Symptom	Checks	Action
UI not reachable	`systemctl status dbaegis`, `ss -ltnp`, nginx logs	Restart service, verify ports `3000` / `3443`, check firewall
HTTPS browser warning	certificate issuer/name trust	Use customer-provided trusted certificate for production

Symptom	Checks	Action
Login fails	bootstrap password, user enabled, session state, logs	Reset admin password only through approved admin flow
API route returns `Not Found` after upgrade	stale backend process	Restart `dbaegis`, hard refresh browser
License invalid	token path, public key, edition, expiry, instance ID	Install matching token/key and restart
Feature locked unexpectedly	active edition and license feature claims	Verify `/api/license/status` and package `release.json`
Backup fails	connection precheck, UI failure guidance, tool path, credentials, storage write access	Use the UI failure panel to identify disk/quota, cloud, auth, SSH/network, missing-tool, timeout, or MySQL physical-safety causes. Fix the cause, open logs if needed, then rerun manual backup.
Restore fails	artifact exists, UI failure guidance, target connection, tool path, permissions	Use the UI failure panel to identify target/config, disk/quota, cloud, auth, SSH/network, missing-tool, or timeout causes. Restore to a validation target when possible, review the job log, then retry.
MySQL or MariaDB physical backup rejects datadir archive	live datadir markers, missing `xtrabackup` / `mariadb-backup`, `consistency` setting	Install the native physical backup tool, or point DBAegis at an offline/frozen snapshot and set `physical_options.consistency=snapshot-safe`
SQL Server or Azure SQL BACPAC export rejects password auth	`sqlpackage` password would be passed as a process argument	Use token/passwordless auth, or set `DBAEGIS_ALLOW_SQLPACKAGE_PASSWORD_ARG=1` only after customer approval
SMTP test fails	SMTP config, CSRF token, auth session, network	Refresh UI, retry, check logs and SMTP reachability
Database engine cannot create restore target	database service datadir/mount health, database permissions, free space	Fix the database engine or container storage first, then rerun the DBAegis restore
Database is locked	duplicate processes, external SQLite access	Stop duplicate processes and restart service
Missing backup artifacts	storage destination, retention, object path	Check backup history, local path, and cloud object logs
`DBAEGIS_SECRET_KEY` error	key changed or missing	Restore old key or re-enter saved secrets

Symptom	Checks	Action
Vendor tool permission issue	`/opt/dbaegis/vendor` ownership and executable bits	Repair service-user ownership and read/execute permissions

Primary logs:

```
sudo journalctl -u dbaegis --since "2 hours ago" --no-pager
sudo -u dbaegis tail -n 200 /opt/dbaegis/logs/dbaegis.log
sudo -u dbaegis tail -n 200 /opt/dbaegis/logs/nginx-error.log
```

Minimum support bundle facts:

- DBAegis version and edition
- license status without exposing token contents
- OS and install path
- relevant service logs with secrets redacted
- database type and backup/restore mode
- storage destination type
- job ID, connection ID, schedule ID, or restore ID
- exact error message and timestamp

Do not ask customers to send private keys, issuer keys, database passwords, cloud secrets, TLS private keys, raw metadata DB files, or unredacted configs.

Operational Validation After Product Changes

After every install, upgrade, downgrade, rollback, restore, or configuration change, verify the service before returning it to normal operation.

```
curl http://127.0.0.1:8000/health
curl http://127.0.0.1:8000/api/version
curl http://127.0.0.1:8000/api/license/status
systemctl status dbaegis --no-pager
```

For paid editions, also verify login, license validity, and at least one licensed backup/restore workflow in a non-destructive test target.

License Renewal And Edition Change

1. Obtain the replacement token and public verification key from the DBAegis license issuer or support contact.
2. Stage the new token and public key on the customer server.
3. Restart DBAegis if the files changed while the service was running.
4. Verify:

```
dbaegis license verify \
  --license-key-file /opt/dbaegis/license/dbaegis.license \
  --public-key-file /opt/dbaegis/license/license_public.pem

curl http://127.0.0.1:8000/api/license/status
```

1. For edition upgrades or downgrades, install the target edition package with `bin/install.sh --upgrade`. Do not change editions by editing only `DBAEGIS_EDITION`.

Support Workflow

1. Capture the environment:

```
curl http://127.0.0.1:8000/api/version
curl http://127.0.0.1:8000/api/license/status
systemctl status dbaegis --no-pager
journalctl -u dbaegis --since "2 hours ago" --no-pager
```

1. Capture the affected edition, database type, storage destination, backup type, restore type, schedule, connection ID, backup ID, and restore job ID.
2. Share redacted logs only. Do not send private keys, license issuer keys, database passwords, cloud secrets, TLS private keys, raw metadata DB files, or unredacted configs through support channels.
3. Reproduce with the narrowest safe path, preferably a non-production target.
4. Preserve backup artifacts and logs until the issue is understood.

Security And Incident Response

Escalate immediately if any of these are suspected:

- issuer private key exposure
- customer license token leakage
- package artifact tampering
- customer data, backup artifact, or metadata DB exposure
- authentication, authorization, or license bypass
- secret redaction failure
- destructive restore executed unexpectedly

Immediate response:

1. Preserve evidence.
2. Stop using or distributing affected artifacts until reviewed.
3. Identify affected editions, hosts, packages, license tokens, databases, and backup artifacts.
4. Rotate affected secrets, credentials, TLS keys, or license tokens when required.
5. Restore from known-good product or self-backup state if needed.
6. Record the incident, impact, customer communication, and remediation.

Monitoring And Operations

At minimum, monitor these product signals:

- service health
- HTTP/API availability
- failed backups and restores
- scheduled-job lag
- license expiration and license invalid state
- metadata DB growth and disk usage
- backup directory usage
- self-backup freshness
- log volume

Run a periodic restore drill. A backup that is never restored is not validated.

Control-Plane Recovery

Use the self-backup and restore procedure in this handbook to recover DBAegis metadata/config state after VM loss, metadata corruption, or operator error.

Recover on a replacement host:

1. Provision a supported VM with DNS, firewall rules, service account model, and storage mounts matching the customer design.
2. Install the same DBAegis release that created the selected self-backup, or install the target release and review release notes for metadata migrations.
3. Stop DBAegis before replacing runtime state.

```
sudo systemctl stop dbaegis
```

1. Restore required files from infrastructure backup or configuration management:

- `/opt/dbaegis/conf/dbaegis.conf` with the correct `DBAEGIS_SECRET_KEY`
- `/opt/dbaegis/license` if Professional or Enterprise licensing is enabled
- `/opt/dbaegis/tls` if local TLS material is used
- `/backups/self` self-backup archives
- `/backups` local database backup artifacts when DBAegis-local storage is used

1. Restore the selected metadata DB from the self-backup archive.

```
snapshot="/backups/self/selfbackup_YYYYMMDD_HHMMSS_NNNNNNNNN.zip"
tmpdir="$(sudo -u dbaegis mktemp -d /opt/dbaegis/tmp/controlplane-restore.XXXXXX)"
sudo -u dbaegis unzip "$snapshot" -d "$tmpdir"
sudo -u dbaegis install -m 640 "$tmpdir/dbaegis.db" /opt/dbaegis/data/dbaegis.db
sudo -u dbaegis rm -rf "$tmpdir"
```

1. Repair ownership and permissions if files were restored by root or backup tooling.

```
sudo chown -R dbaegis:dbaegis /opt/dbaegis/data /opt/dbaegis/conf /opt/dbaegis/logs
/opt/dbaegis/tmp /backups
sudo chmod 640 /opt/dbaegis/data/dbaegis.db /opt/dbaegis/conf/dbaegis.conf
```

1. Start DBAegis and run post-restore validation.

```
sudo systemctl start dbaegis
sudo systemctl status dbaegis --no-pager
curl http://127.0.0.1:8000/health
curl http://127.0.0.1:8000/api/version
curl http://127.0.0.1:8000/api/license/status
```

If the matching `DBAEGIS_SECRET_KEY` is lost, the metadata and history can still be restored, but saved connection passwords, storage credentials, SMTP passwords, LDAP bind credentials, MFA enrollments, webhook secrets, and sensitive restore options must be re-entered before affected workflows can run. Take a new self-backup after re-entry and store the active key off-host.

Minimum recurring validation:

- self-backup creation
- off-host copy of self-backup artifacts
- restore into a clean environment
- verification that saved secrets decrypt with the matching `DBAEGIS_SECRET_KEY`
- login, license, schedule/history visibility, and non-destructive backup/restore smoke after recovery

Dependency And Platform Maintenance

For packaged customer installs, review DBAegis release notes, dependency advisories, and customer support notices before upgrading. Optional vendor tools such as SnowSQL, SqlPackage, MongoDB tools, ClickHouse client, and database-native backup tools should be patched according to customer security policy and then validated with a safe backup/restore test.

Browser And UI Compatibility

For every customer-facing UI release:

- test the current Chrome, Edge, Firefox, and Safari families where possible
- verify login/logout, dashboard refresh, connections, schedules, backup history, restore jobs, license banners, edition-locked panels, and access-control views
- test at desktop and narrow viewport widths
- verify HTTP and configured HTTPS behavior

Record browser compatibility evidence with the release notes or validation artifact for the target release.

Customer Maintenance Cadence

Cadence	Required Work
Every change	service health check, login check, license check, affected workflow test
Weekly	failed backup/restore review, schedule lag review, disk usage review
Monthly	restore drill, self-backup restore review, browser/UI smoke
Quarterly	control-plane DR drill, secret/key handling review, access-control review
Before major upgrade	config/DB safety copy, edition/license check, rollback plan, post-upgrade smoke